

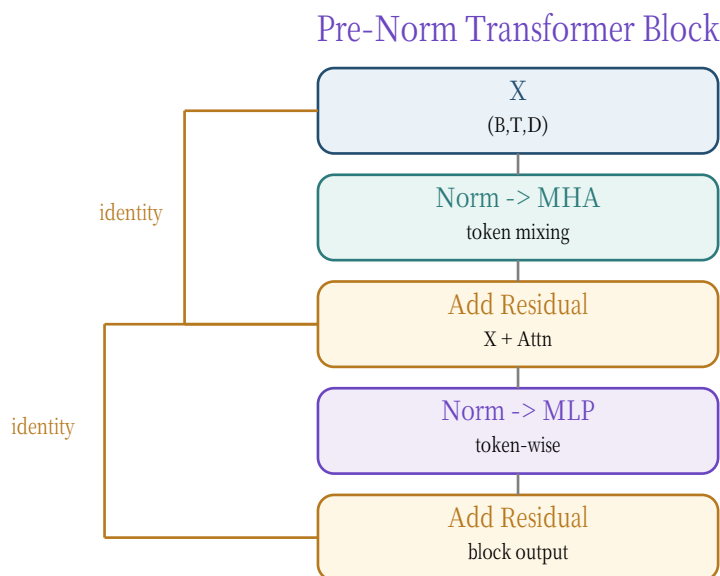
## 第 56 讲

# Transformer 中的 MLP、Residual、LayerNorm / RMSNorm

## 怎样把 Attention 装回完整 Transformer Block

### 本讲定位

第 53-55 讲已经把 Q/K/V、Scaled Dot-Product Attention、Multi-Head Attention 拆开。本讲不再增加新的大模块，而是把你之前学过的 MLP、Residual、Normalization 与 Attention 重新装成一个完整 Block。



| 模块                   | 主要职责                           |
|----------------------|--------------------------------|
| Multi-Head Attention | Token 与 Token 之间的信息交互          |
| MLP / FFN            | 每个 Token 内部的特征变换               |
| Residual             | 提供 identity path, 保留原表示并改善深层优化 |
| LayerNorm / RMSNorm  | 控制每个 Token hidden vector 的尺度   |

# 1. 为什么只有 Attention 还不够

Attention 很擅长回答“当前 Token 应该从哪些 Token 读取信息”，但读取完成后，还需要进一步对当前 Token 的特征进行非线性变换。这个职责通常由 MLP / FFN 完成。

|                                               |
|-----------------------------------------------|
| Attention: token mixing                       |
| MLP: feature transformation within each token |

因此一个 Transformer Block 的核心分工可以先粗略理解成：

| 问题                   | 主要模块                                                                                              |
|----------------------|---------------------------------------------------------------------------------------------------|
| 我应该从哪些 Token 读取信息？   | Attention                                                                                         |
| 读到以后，怎样重新组合/加工自己的特征？ | MLP                                                                                               |
| 怎样不把原来的信息完全覆盖掉？      | Residual                                                                                          |
| 怎样维持深层网络数值尺度可控？      | Norm                                                                                              |
| 不要把 MLP 当成附属品        | 现代 Transformer Block 中，MLP 往往占据非常大的 Parameter 与 FLOP 比例。Attention 负责交互，MLP 负责每个 Token 内部的大量非线性计算。 |

## 2. 一个 Pre-Norm Block 的完整公式

典型 Pre-Norm 写法可以拆成两段：

$$X_1 = X + \text{Attention}(\text{Norm}_1(X))$$

$$Y = X_1 + \text{MLP}(\text{Norm}_2(X_1))$$

如果使用 RMSNorm，只是把 Norm\_1 / Norm\_2 的具体归一化方式从 LayerNorm 换成 RMSNorm，整体 Block 拓扑不变。

这两条公式有一个非常重要的共同结构：

$$\text{output} = \text{input} + \text{transformed}(\text{input})$$

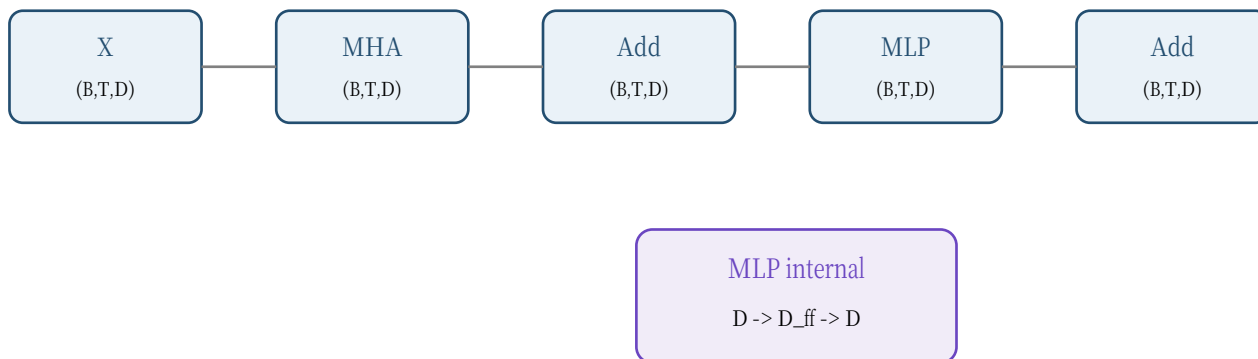
也就是 Residual Connection。

### Shape 要求

因为要做逐元素相加，Residual 主路径和子层输出必须拥有相同 Shape。标准 Transformer 中这就是为什么 Attention 和 MLP 子层最终都要回到 (B,T,D\_model)。

### 3. Shape 主线：Block 外部几乎一直是 (B,T,D)

Block 外部 Shape 通常始终保持 (B,T,D)



虽然内部 Attention 会拆 Head、MLP 会把 D 扩张到  $D_{ff}$ ，但每个子层回到 Residual Add 前，都必须重新恢复：

(B,T, $D_{model}$ )

例如：

MHA: (B,T,768)  $\rightarrow$  internal heads  $\rightarrow$  (B,T,768)

MLP: (B,T,768)  $\rightarrow$  (B,T,3072)  $\rightarrow$  (B,T,768)

因此堆很多 Transformer Block 时，Residual Stream 的主 Shape 可以长期保持不变。

#### 读源码技巧

看到一个复杂 Block 时，先沿 residual stream 追 (B,T,D)。内部 head\_dim、 $D_{ff}$  都是暂时展开，回到 Add 前通常会收回  $D_{model}$ 。

# 4. MLP / FFN：为什么先扩张再压回去

经典 FFN：

$$\text{MLP}(x) = W_2 * \text{phi}(W_1 x + b_1) + b_2$$

典型 Shape：

$$D_{\text{model}} \rightarrow D_{\text{ff}} \rightarrow D_{\text{model}}$$

例如：

$$768 \rightarrow 3072 \rightarrow 768$$

中间维度  $D_{\text{ff}}$  比  $D_{\text{model}}$  大，让模型在更宽的非线性特征空间中进行变换，然后再投影回 Residual Stream 的  $D_{\text{model}}$ 。

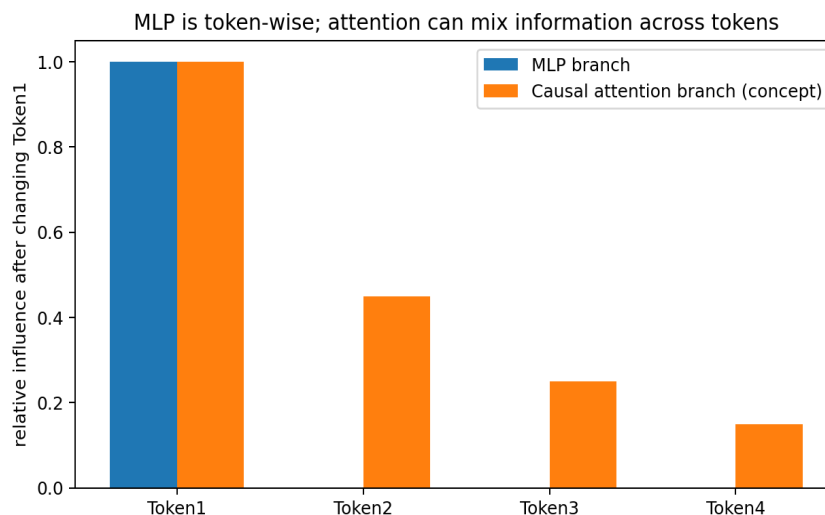
| 阶段                   | Shape                                                                                          | 作用                    |
|----------------------|------------------------------------------------------------------------------------------------|-----------------------|
| Linear 1             | $(B,T,D) \rightarrow (B,T,D_{\text{ff}})$                                                      | 扩张特征空间                |
| GELU / SiLU / SwiGLU | 保持外部 Shape                                                                                     | 引入非线性/门控              |
| Linear 2             | $(B,T,D_{\text{ff}}) \rightarrow (B,T,D)$                                                      | 压回 residual dimension |
| 与第 9-10 讲连接          | Transformer 的 FFN/MLP 没有新数学：仍然是 Linear + Activation + Linear。只是现在应用在每个 Token 的 hidden state 上。 |                       |

## 5. MLP 不会做 Token Mixing

标准 MLP 对最后一个特征维执行同一套映射：

$$y[b,t,:] = \text{MLP}(x[b,t,:])$$

Token  $t$  的 MLP 输出不会直接读取 Token  $s$  的 hidden vector。实验中只修改 Token1，MLP 输出只有 Token1 发生变化，其余 Token 输出完全不变。



相比之下，Causal Attention 中修改更早的 Token1，可以影响 Token2、Token3、Token4，因为后续 Query 允许读取 Token1 的 K/V。

### Transformer Block 的分工

Attention = token mixing；MLP = token-wise feature mixing。这是理解 Block 机制最重要的二分之一。

## 6. Residual Connection 到底为什么重要

$$y = x + F(x)$$

最直观的作用是：子层不必从零“重新生成”整个表示，只需要学习在原表示基础上增加一个增量。

如果  $F(x)=0$ ：

$$y=x$$

因此 Residual 提供了一条 identity

path。深层网络即使某一层暂时没有学到有用变换，也可以近似把输入原样传过去。

### 优化直觉

没有 Residual 时，每一层都必须通过自己的变换才能把信息传到下一层；有 Residual 时，信息和 Gradient 多了一条直接路径。这是深层 Transformer 能稳定堆叠的重要结构之一。

## 7. Residual 的 Gradient 为什么更容易传播

对：

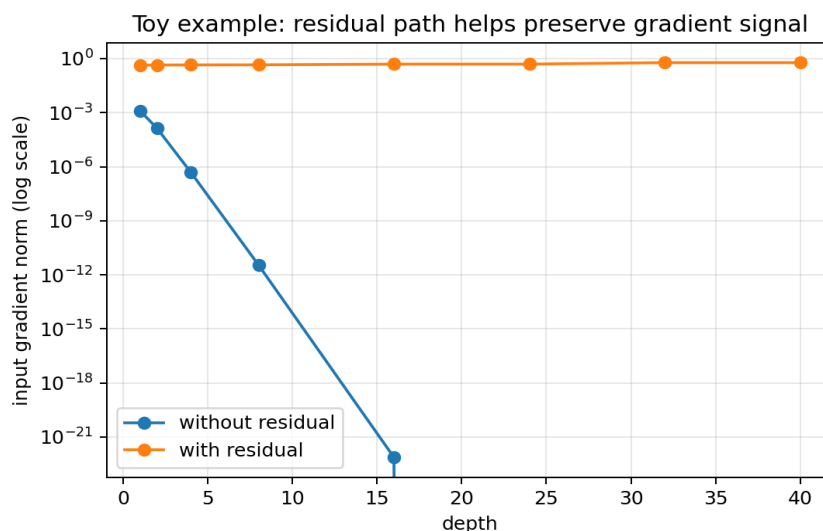
$$y = x + F(x)$$

Jacobian 可以写成：

$$dy/dx = I + dF/dx$$

关键是出现了 Identity Matrix  $I$ 。即使  $dF/dx$  很小，仍然存在一条直接梯度路径。

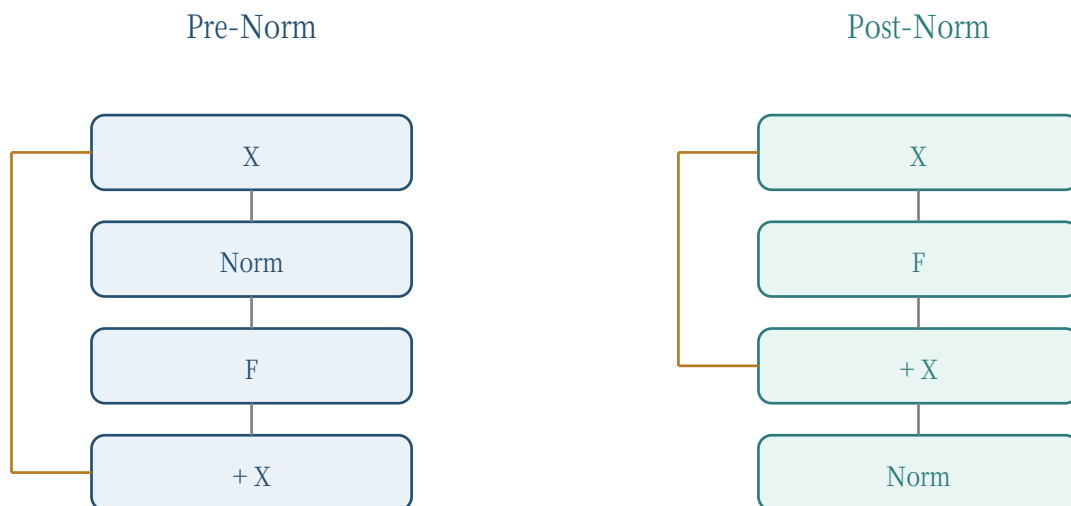
本讲 toy 实验用相同随机非线性层堆到几十层，对比有无 Residual 的输入 Gradient Norm。没有 Residual 时梯度迅速变得极小，而加入 Residual 后仍维持可传播的数量级。



边界

Residual 不能保证任何深层网络都稳定，也不能代替合理初始化、Normalization、Learning Rate 等。它只是显著改善深层优化条件。

## 8. Pre-Norm 与 Post-Norm：不是同一句公式换顺序



关键区别：Pre-Norm 的 residual 输出可以直接保留 X；Post-Norm 最后仍要经过 Norm

$$\text{Pre-Norm: } y = x + F(\text{Norm}(x))$$

$$\text{Post-Norm: } y = \text{Norm}(x + F(x))$$

Pre-Norm 中，identity path 从 x 直接到 Add，不需要先穿过 Norm；Post-Norm 中，Residual Add 之后仍然必须经过 Norm。

Pre-Norm

$$y = x + F(\text{Norm}(x))$$

identity path reaches add directly

Post-Norm

$$y = \text{Norm}(x + F(x))$$

sum still passes through Norm

This structural distinction affects optimization and gradient flow; it is not just notation.

### 现代 LLM 连接

很多现代大模型采用 Pre-Norm 风格，并常配 RMSNorm。原始 2017 Transformer 则使用 Post-Norm。二者的差异与深层训练稳定性密切相关。

## 9. 一个极小的“F=0”实验看 Pre/Post 差别

我们把 Attention 和 MLP 所有可训练 Weight 都清零，让子层输出严格为 0。

Pre-Norm：

$$y = x + 0 \rightarrow x$$

实验得到：

$$\max |y_{\text{pre}} - x| = 0$$

Post-Norm：

$$y = \text{Norm}(x + 0) = \text{Norm}(x)$$

因此一般：

$$y_{\text{post}} \neq x$$

这个极端 toy case 直接展示了结构差异：Pre-Norm 的 residual stream 保留真正的 identity path；Post-Norm 最终输出仍受 Norm 变换。

**不要过度外推**

这并不意味着 Post-Norm 一定训练失败。它只是说明两种拓扑的梯度/信号路径结构不同。真实训练效果还受深度、初始化、优化器、LR、残差缩放等多因素影响。

# 10. LayerNorm / RMSNorm 在 Block 中到底做什么

你在第 47 讲已经学过：典型 Transformer 中 Norm 对每个 Token 自己的 hidden dimension 做尺度控制。

X:(B,T,D) -> Norm over D

它不负责 Token mixing，也不改变 T。它的主要作用是让送入 Attention / MLP 的 hidden vector 处于更可控的尺度范围。

| Norm      | 是否减均值 | 典型可学习参数      |
|-----------|-------|--------------|
| LayerNorm | 是     | gamma + beta |
| RMSNorm   | 否     | 通常 gamma     |

在 Pre-Norm Block 中，Residual Stream 本身可以保留未归一化的主路径，而子层输入先经过 Norm。

# 11. Dropout 通常插在哪里

不同实现略有差异，但常见位置包括：

| 位置                            | 作用                       |
|-------------------------------|--------------------------|
| Attention weights 上           | Attention Dropout        |
| Attention output projection 后 | 对子层输出正则化                 |
| MLP 中间或输出后                    | MLP Dropout              |
| Residual Add 前                | 让随机失活作用在 residual branch |

因此 `model.train()` / `model.eval()` 会影响含 Dropout 的 Transformer Block，而 LayerNorm / RMSNorm 本身 train/eval 行为通常一致。

本讲实验使用 dropout=0.5 验证：train 模式两次 forward 输出不同；eval 模式两次 forward 完全一致。

评估协议

验证/测试时忘记 `model.eval()`，Dropout 仍在随机失活，会让结果带额外随机性。这一点与第 44 讲直接相连。

## 12. 一个 Tiny Pre-Norm Block 的完整 Forward

```
a_in = norm1(x)
a = mha(a_in, a_in, a_in, attn_mask=mask)
x = x + a

m_in = norm2(x)
m = fc2(gelu(fc1(m_in)))
x = x + m
```

实验中：

`x.shape = (2,4,8)`

Attention、Residual、Norm、MLP 经过每一步以后，Block 外部始终恢复为：

`(2,4,8)`

只有 fc1 暂时扩成：

`(2,4,16)`

然后 fc2 再压回：

`(2,4,8)`

手工逐步 forward 与封装的 `TinyPreNormBlock.forward()` 最大误差为 0。

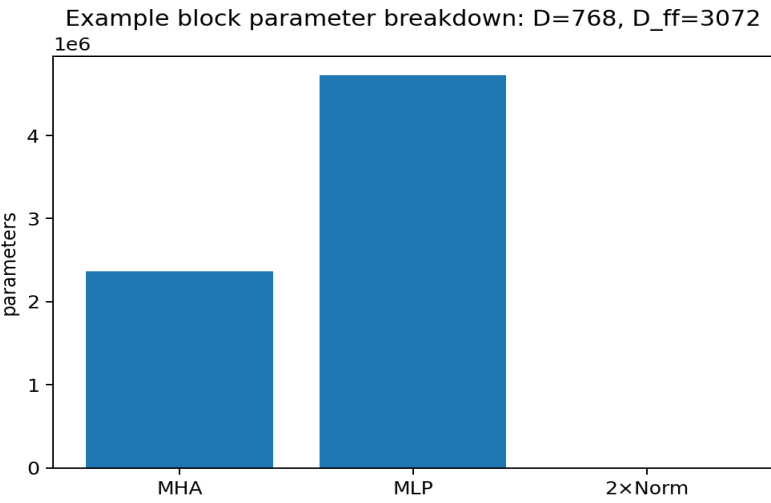
# 13. Transformer Block 中到底谁负责参数量

取一个经典教学配置：

D\_model=768, D\_ff=3072, H=12

实验统计 PyTorch 模块参数量：

| 模块                   | 参数量       |
|----------------------|-----------|
| Multi-Head Attention | 2,362,368 |
| MLP                  | 4,722,432 |
| 2 × LayerNorm        | 3,072     |
| Residual             | 0         |



为什么 MLP 很重要

这个配置里，MLP 参数量大约是 MHA 的两倍。只盯着 Attention 容易低估 FFN/MLP 在 Transformer 中的规模和计算作用。

## 14. 参数量公式：以后读论文可以自己估

标准 MHA（忽略 bias）大约：

$$Q, K, V, O \rightarrow 4D^2$$

经典两层 MLP（忽略 bias）：

$$D \rightarrow D_{ff} \rightarrow D \Rightarrow 2D * D_{ff}$$

若  $D_{ff} \approx 4D$ ：

$$MLP \approx 8D^2$$

于是一个经典 Block 粗略：

$$MHA + MLP \approx 12D^2$$

Norm 参数与这些大矩阵相比通常很小。

### 现代变体

使用 SwiGLU 等 gated MLP 时参数公式会改变；GQA/MQA 也会改变 Attention 参数与 KV 结构。读具体模型时要按实际 Weight Shape 计算。

## 15. Residual 为什么要求维度对齐

$$x + F(x)$$

要求  $x$  与  $F(x)$  Shape 一致。标准 Transformer 因此把 Attention 和 MLP 最终都投影回  $D_{\text{model}}$ 。

如果主分支维度改变，理论上可以像 ResNet 那样对 shortcut 加 Projection，但标准语言 Transformer Block 通常保持 residual dimension 不变。

对源码的一个检查法

看到 Block 中某个子层输出不是  $D_{\text{model}}$  时，继续找后面的 output projection；如果最终还没恢复  $D_{\text{model}}$ ，就无法直接与 residual stream 相加。

# 16. Attention 与 MLP 的 “ 分工 ” 为什么互补

| 模块       | 跨 Token? | 非线性?           | 主要作用               |
|----------|----------|----------------|--------------------|
| MHA      | 是        | Softmax 带非线性结构 | 动态聚合其他 Token 信息    |
| MLP      | 否        | 是              | 每个 Token 内部重组与扩展特征 |
| Residual | 不单独计算新关系 | 否              | 保留/累加表示            |
| Norm     | 组内特征耦合   | 归一化变换          | 控制尺度               |

可以把一个 Block 粗略理解成：

先交流，再独立思考；每一步都保留原来的 residual stream

只是比喻

“ 交流/思考 ” 有助于建立结构直觉，但模型内部仍然只是 Tensor 运算，不要把这种比喻当成人类认知机制等价。

# 17. Pre-Norm / Post-Norm 做实验时怎样公平比较

如果论文要比较 Pre-Norm 与 Post-Norm，应至少控制：

| 控制项                                 | 原因                                                             |
|-------------------------------------|----------------------------------------------------------------|
| 模型深度/宽度                             | 避免容量差异                                                         |
| Attention / MLP Weight 初始化          | 避免起点差异                                                         |
| Optimizer / LR / Scheduler / Warmup | 两种结构最优优化超参可能不同                                                 |
| 训练 Steps / Tokens                   | 保证预算一致                                                         |
| 随机种子                                | 优化稳定性本身可能有方差                                                   |
| 最终 Test 协议                          | 避免只比较 Train Loss                                               |
| 研究注意                                | 如果一种结构需要不同 warmup/LR 才稳定，那么“统一超参”与“各自调优”回答的是不同科研问题。论文必须说明比较口径。 |

# 18. 与 GPT / LLM 源码怎样对应

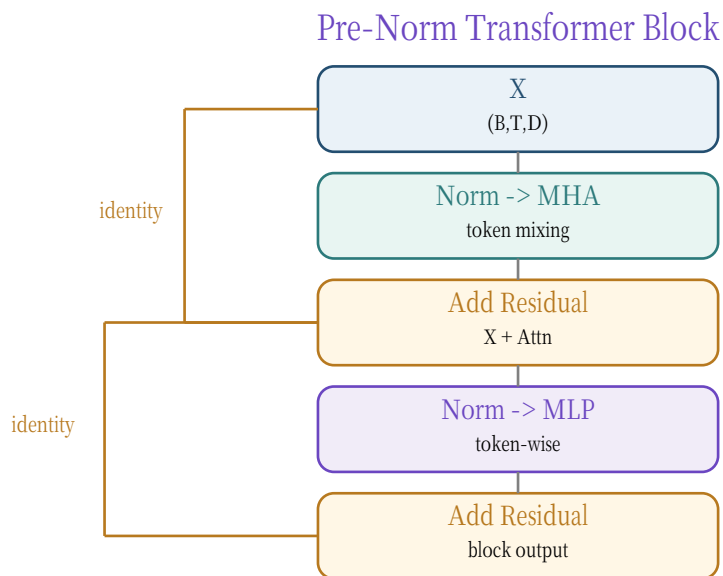
| 源码名                             | 本讲对应                                 |
|---------------------------------|--------------------------------------|
| input_layernorm / ln_1          | Attention 前 Norm                     |
| self_attn                       | Multi-Head Self-Attention            |
| residual add                    | $x = residual + attn\_out$           |
| post_attention_layernorm / ln_2 | MLP 前 Norm                           |
| mlp / feed_forward              | $D \rightarrow D_{ff} \rightarrow D$ |
| down_proj / up_proj / gate_proj | 现代 gated MLP 的不同线性投影                 |
| o_proj                          | Attention concat 后 Output Projection |

以后读开源 LLM 时，可以先忽略所有优化细节，按这张表把 Block 还原为：Norm -> Attention -> Add -> Norm -> MLP -> Add。

Agent Systems  
连接

你做 Agent Systems 研究未必修改 Block，但理解这一层结构能帮助你判断模型差异、LoRA target modules、长上下文机制和底座模型训练稳定性。

## 19. 本讲总装与 5 分钟自检



请尝试口述：

- 1) 为什么 Attention 之后还需要 MLP？
- 2) 为什么说 Attention 负责 token mixing，而 MLP 通常不负责？
- 3) Residual 为什么能提供 identity path？
- 4) Pre-Norm 与 Post-Norm 的公式分别是什么？
- 5) 为什么 MHA / MLP 最终都必须回到  $D_{\text{model}}$ ？
- 6) MLP 内部为什么常见  $D \rightarrow D_{\text{ff}} \rightarrow D$ ？
- 7) 一个典型 Block 中，MHA 与 MLP 谁往往参数更多？
- 8) train/eval 对 LayerNorm 和 Dropout 的影响有什么不同？

下一讲

第 57 讲会把 Token Embedding、位置机制、Multi-Head Attention、Norm、MLP、Residual 串成完整 Transformer Block / Decoder 数据流，并逐层追踪从输入 Token 到 Block 输出的每一个 Shape。